

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
def predict(X, weights, lambd=1):
    net = np.dot(X, weights)
    return (1 / (1 + np.exp(-lambd * net)))

def calculate_dw(X, y, y_pred):
    return ((y_pred - y) * (y_pred) * (1 - y_pred) * X)

def calculate_error(yi, y_pred):
    return np.square(y_pred - yi)
```

```
def train_vanilla(X, y, weights, epochs=500, lr=0.01, epsilon=1e-8, beta1=0.9, beta2=0.999):
    mom = 0
    vel = 0

    for epoch in range(epochs):
        dw = 0
        error = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)

        mom = (beta1 * mom) + ((1 - beta1) * dw)
        vel = (beta2 * vel) + ((1 - beta2) * (dw**2))
        step = epoch + 1
        momcap = mom / (1 - (beta1**step))
        velcap = vel / (1 - (beta2**step))
        weights -= ((lr * momcap) / (np.sqrt(velcap + epsilon)))
        error /= (2 * len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_stochastic(X, y, weights, epochs=500, lr=0.01, epsilon=1e-8, beta1=0.9, beta2=0.999):
    mom = 0
    vel = 0
    step = 0

    for epoch in range(epochs):
        error = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw = calculate_dw(Xi, yi, y_pred)
            error += calculate_error(yi, y_pred)
            mom = (beta1 * mom) + ((1 - beta1) * dw)
            vel = (beta2 * vel) + ((1 - beta2) * (dw**2))
            step += 1
            momcap = mom / (1 - (beta1**step))
            velcap = vel / (1 - (beta2**step))
            weights -= ((lr * momcap) / (np.sqrt(velcap + epsilon)))

        error /= (2 * len(X))

        if (epoch+1)%50 == 0:
            print(f'Weights after epoch {epoch+1} : ',weights)
            print(f'Error after epoch {epoch+1} : ',error)
```

```
def train_mini_batch(X, y, weights, epochs=500, lr=0.01, epsilon=1e-8, beta1=0.9, beta2=0.999, bs=32):
    mom = 0
    vel = 0
    step = 0

    for epoch in range(epochs):
        error = 0
        dw = 0
        i = 0

        for Xi, yi in zip(X, y):
            y_pred = predict(Xi, weights)
            dw += calculate_dw(Xi, yi, y_pred)
```

```

    i += 1
    error += calculate_error(yi, y_pred)

    if i%bs == 0 or i == len(X):
        mom = (beta1 * mom) + ((1 - beta1) * dw)
        vel = (beta2 * vel) + ((1 - beta2) * (dw**2))
        step += 1
        momcap = mom / (1 - (beta1**step))
        velcap = vel / (1 - (beta2**step))
        weights -= ((lr * momcap) / (np.sqrt(velcap + epsilon)))
        dw = 0
    error /= (2 * len(X))

    if (epoch+1)%50 == 0:
        print(f'Weights after epoch {epoch+1} : ',weights)
        print(f'Error after epoch {epoch+1} : ',error)

```

```

df = pd.read_csv('bank_note.csv')
df.insert(4, 'x0', 1)

```

```

X = df[['variance', 'skewness', 'curtosis', 'entropy', 'x0']].values
y = df['class']

```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=13, test_size=0.2)
weights = input('Enter 4 weights and 1 bias : ').split()
weights = np.array([float(weight) for weight in weights], dtype='longdouble')

```

```

print('Adam (Vanilla)')
train_vanilla(X_train, y_train, weights.copy())

```

```

Adam (Vanilla)
Weights after epoch 50 : [ 0.01585252 -0.48583405  0.24332349 -0.80139811  0.44419424]
Error after epoch 50 : 0.10702361087191543
Weights after epoch 100 : [-0.33940647 -0.51950831 -0.2649651 -0.71347022  0.49268423]
Error after epoch 100 : 0.04372105121775649
Weights after epoch 150 : [-0.60203996 -0.51101469 -0.48411533 -0.33039449  0.75317172]
Error after epoch 150 : 0.01796901131387408
Weights after epoch 200 : [-0.75989198 -0.52242694 -0.55695582 -0.15484596  1.1010441 ]
Error after epoch 200 : 0.012057094797969023
Weights after epoch 250 : [-0.86916062 -0.54894454 -0.61449686 -0.07761004  1.30637961]
Error after epoch 250 : 0.009852392747750576
Weights after epoch 300 : [-0.95509729 -0.58122165 -0.66634952 -0.03989542  1.45060131]
Error after epoch 300 : 0.008647057394740282
Weights after epoch 350 : [-1.02744948 -0.61449002 -0.71374958 -0.0194097  1.56051837]
Error after epoch 350 : 0.007858963567316095
Weights after epoch 400 : [-1.09094327 -0.64684382 -0.75722211 -0.00760367  1.64867009]
Error after epoch 400 : 0.007294099271211063
Weights after epoch 450 : [-1.14818115e+00 -6.77675916e-01 -7.97322839e-01 -6.84304994e-04
 1.72186709e+00]
Error after epoch 450 : 0.006865654954697856
Weights after epoch 500 : [-1.20074398 -0.70687837 -0.8345699  0.00325253  1.78424411]
Error after epoch 500 : 0.006527598316542039

```

```

print('Adam (Stochastic)')
train_stochastic(X_train, y_train, weights.copy())

```

```

Adam (Stochastic)
Weights after epoch 50 : [-4.62802048 -2.32821612 -2.97498474  0.13987812  4.5108557 ]
Error after epoch 50 : 0.0041023592001994255
Weights after epoch 100 : [-6.01427437 -3.02438737 -3.89074668  0.17018931  5.72789928]
Error after epoch 100 : 0.003908207602572633
Weights after epoch 150 : [-7.02238674 -3.54279239 -4.56128574  0.16388196  6.67713625]
Error after epoch 150 : 0.003831436359544577
Weights after epoch 200 : [-7.87584457 -3.98523274 -5.13149363  0.11117256  7.42067564]
Error after epoch 200 : 0.0038311496147869877
Weights after epoch 250 : [-8.64890477 -4.38688 -5.6450762  0.0505421  8.06716124]
Error after epoch 250 : 0.0038401962341422864
Weights after epoch 300 : [-9.36141378 -4.74280533 -6.09986631  0.01933647  8.71148906]
Error after epoch 300 : 0.0038063499390327674
Weights after epoch 350 : [-10.3196006 -5.00590074 -6.35459721  0.01702268  9.74058334]
Error after epoch 350 : 0.003762037115553857
Weights after epoch 400 : [-11.33591423 -5.25555815 -6.61864821 -0.12054972  9.97498622]
Error after epoch 400 : 0.004216756477551327
Weights after epoch 450 : [-11.81848029 -5.50851578 -6.98745132 -0.08954771  10.89465678]
Error after epoch 450 : 0.004088674851684614
Weights after epoch 500 : [-12.09089101 -5.78964589 -7.42790101 -0.09259638  11.39574603]
Error after epoch 500 : 0.0037500085302667745

```

```

print('Adam (Mini Batch)')
train_mini_batch(X_train, y_train, weights.copy())

```



Adam (Mini Batch)

```
Weights after epoch 50 : [-1.85085395 -1.0331295 -1.25164453 -0.0080995  2.29785305]
Error after epoch 50 :  0.004786800573041489
Weights after epoch 100 : [-2.47658128 -1.32479638 -1.64029842 -0.04145616  2.73736286]
Error after epoch 100 :  0.004197142863617189
Weights after epoch 150 : [-2.90865464 -1.52125456 -1.90693674 -0.06493766  3.04716813]
Error after epoch 150 :  0.00397468176816027
Weights after epoch 200 : [-3.22410869 -1.66793209 -2.10497454 -0.08404852  3.28316989]
Error after epoch 200 :  0.0038566579591723634
Weights after epoch 250 : [-3.47151737 -1.78404328 -2.26141241 -0.09930185  3.47308627]
Error after epoch 250 :  0.0037845256471140316
Weights after epoch 300 : [-3.67597074 -1.88027073 -2.39104553 -0.11164548  3.63275395]
Error after epoch 300 :  0.003736261697266815
Weights after epoch 350 : [-3.85074087 -1.96262676 -2.50203302 -0.12190617  3.77098825]
Error after epoch 350 :  0.0037018665867128704
Weights after epoch 400 : [-4.00358749 -2.03470825 -2.59921547 -0.13064665  3.89308398]
Error after epoch 400 :  0.003676233860679297
Weights after epoch 450 : [-4.13945829 -2.09882647 -2.68569375 -0.13824135  4.00248278]
Error after epoch 450 :  0.0036564923357319886
Weights after epoch 500 : [-4.26172113 -2.15655936 -2.76358244 -0.14494647  4.10156514]
Error after epoch 500 :  0.003640900281578409
```